

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Actividades y ciclo de vida



Universidad
Rey Juan Carlos

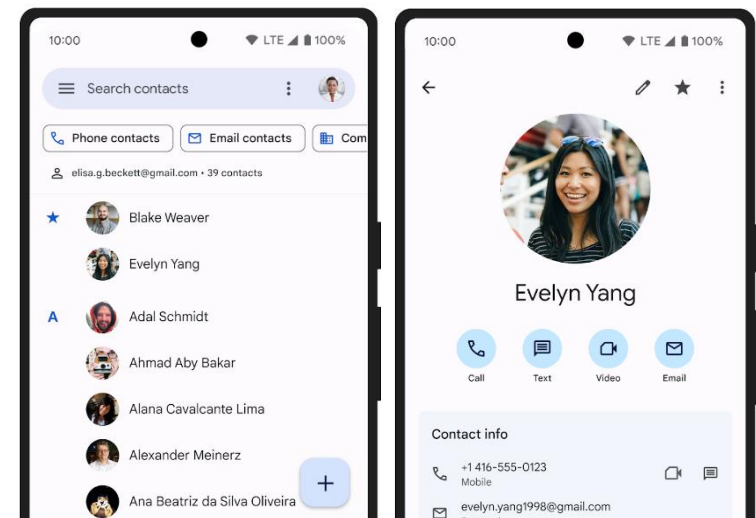
Actividades en Android

La Actividad es un componente fundamental de las aplicaciones Android. **Una actividad proporciona la ventana en la que la app dibuja su IU y con la que el usuario interactúa.** Esta ventana suele ocupar toda la pantalla, aunque puede ser más pequeña que ella y flotar sobre otras ventanas.

Por lo general, una actividad representa una pantalla en una aplicación, y se compone de una **parte gráfica** (xml/compose) y de **una lógica** (kotlin/java).

Una actividad de agenda tendría:

- Una actividad para la lista de contactos
- Una actividad para mostrar un contacto
- Una actividad para crear un contacto
- Una actividad para las preferencias



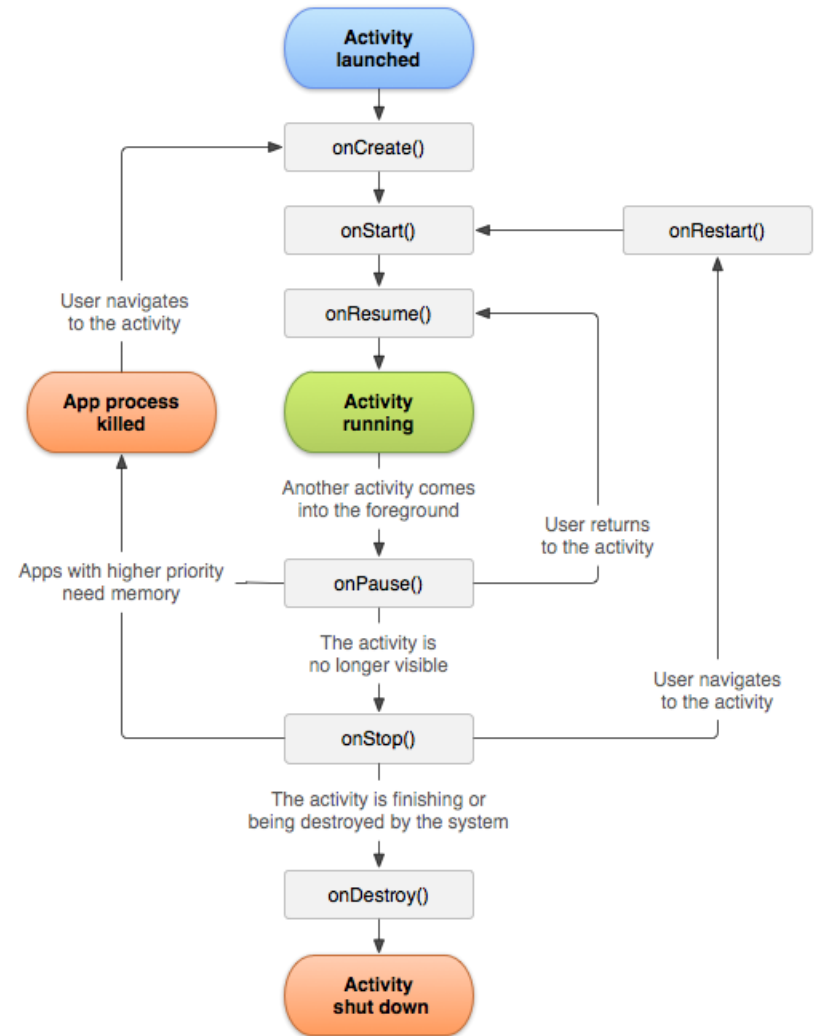
Ciclo de vida de una actividad

Las actividades de una app pasan por diferentes estados de su ciclo de vida.

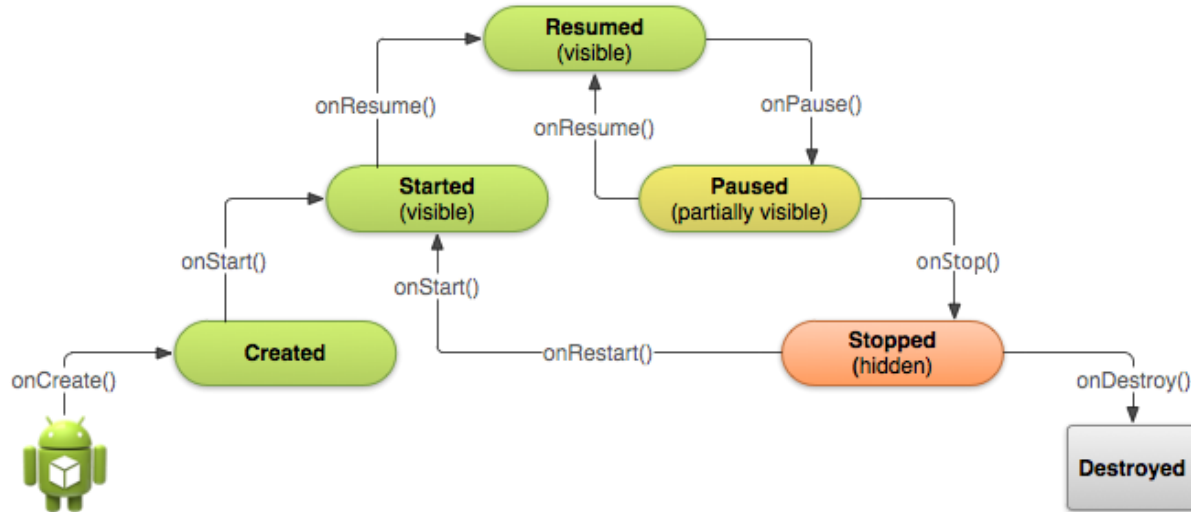
La clase Activity proporciona una serie de **callbacks** que le **informan a la actividad cuándo cambia un estado**.

Estos *callbacks* se pueden sobrescribir para evitar fugas de memoria y fallos de la aplicación y conservar los datos y el estado del usuario si, por ejemplo:

- El usuario abandona temporalmente la aplicación y luego regresa.
- El usuario es interrumpido por una llamada telefónica.
- El usuario gira el dispositivo.



Callbacks y estados de una actividad



Callbacks	Estado al que pasa	Descripción
<code>onCreate()</code>	Created	La actividad está siendo inicializada
<code>onStart()</code>	Started	La actividad es visible al usuario
<code>onResume()</code>	Resumed	La actividad tiene el foco de entrada
<code>onPause()</code>	Paused	La actividad no tiene el foco
<code>onStop()</code>	Stopped	La actividad no está visible
<code>onDestroy()</code>	Destroyed	La actividad se ha destruido

Callbacks del ciclo de vida

onCreate(): se llama al crear la actividad. Es donde se prepara la interfaz gráfica de la pantalla. Aquí se realiza la lógica de inicio básica de la aplicación, que ocurre una sola vez en toda el ciclo de vida. El siguiente método es onStart().

onStart(): se ejecuta justo antes de que la aplicación sea visible al usuario. El siguiente método de ciclo de vida llamado será onStop() u onResume().

onResume(): se ejecuta en el momento en que la actividad se encuentra en la parte superior de la pila, justo antes de que el usuario pueda interactuar con ella. El siguiente método será onPause().

onPause(): se llama cuando la actividad deja de estar en primer plano, aunque seguirá visible si el usuario está en el modo multiventana. Aquí debemos aprovechar para pausar o ajustar las operaciones que no sean necesarias mientras la actividad está pausada. También se pueden liberar recursos del sistema, sensores, etc. El método siguiente será onResume() u onStop().

Callbacks del ciclo de vida (*cont*)

onStop(): se ejecuta cuando la actividad se hace invisible al usuario. Puede ser porque otra actividad la tape o porque vaya a ser destruida. Se usa para liberar o ajustar los recursos que no se necesitan mientras la app no es visible para el usuario. El siguiente método puede ser `onRestart()` u `onDestroy()`.

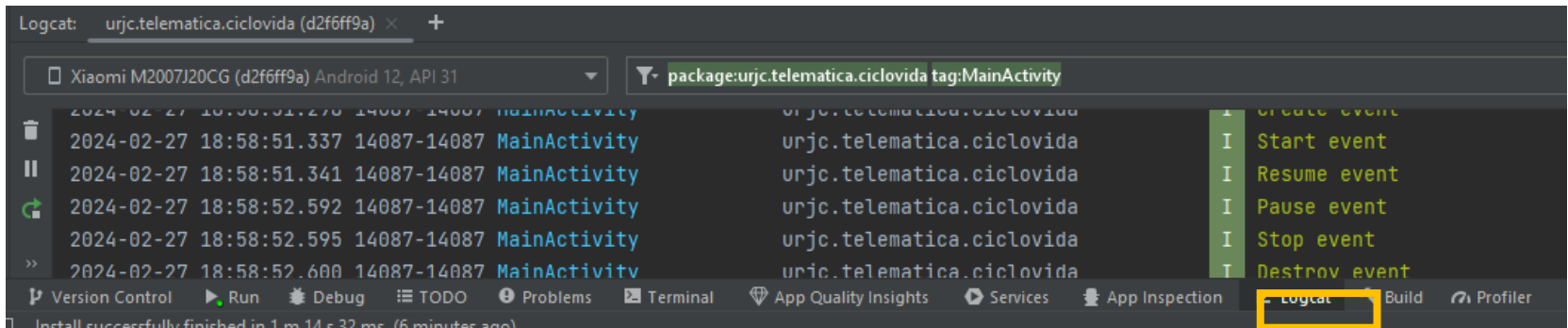
onDestroy(): se llama antes de destruir la actividad. Durante la destrucción se pierden todos los datos asociados a ella, por lo que puede aprovecharse para controlar la persistencia de datos. Se da cuando se ejecuta *finish()* o porque el sistema elimina la actividad para conseguir más recursos.

onRestart(): se llama cuando una actividad que se había parado vuelve a estar activa, justo antes de que comience de nuevo. Tras ella se llama a `onStart()`.

Logging

Clase Log

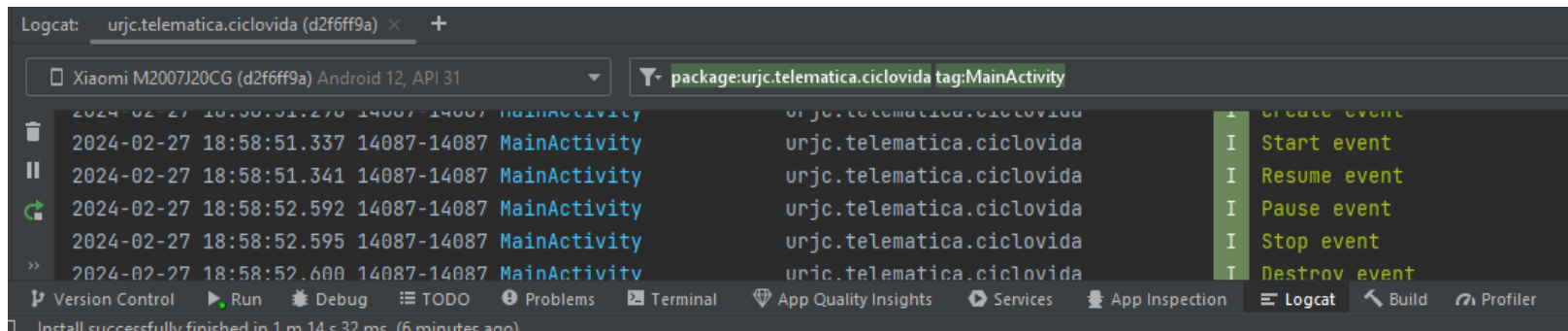
Monitorización de eventos o el estado de la app. Ejemplo: `Log.d("TAG", "Mensaje")`



Nivel de prioridad	Función de Log
Verbose	<i>Log.v(String, String)</i>
Debug	<i>Log.d(String, String)</i>
Info	<i>Log.i(String, String)</i>
Warning	<i>Log.w(String, String)</i>
Error	<i>Log.e(String, String)</i>

Ejercicio 1

1. Crea una aplicación nueva
2. En la actividad principal, sobrescribe todas las callback de cambio de estado de la actividad y añade un mensaje de log a cada una de ellas.
3. Lanza la aplicación en tu móvil / simulador y observa en el *Logcat* cuando se producen las llamadas a cada callback.



Intents

Múltiples actividades

Como hemos visto, a veces **la funcionalidad de la aplicación puede estar separada en varias pantallas.**

Ejemplos:

- Ver los detalles de un único elemento (una peli en una plataforma)
- Crear un nuevo elemento (por ejemplo, un nuevo correo electrónico)
- Mostrar la configuración de una aplicación
- Acceder a servicios de otras aplicaciones (por ejemplo, galería de fotos o examinar documentos)

Intents

Android utiliza una construcción llamada *Intent* para **solicitar una acción de otro componente**.

Un *Intent* puede utilizarse para especificar una **petición de transición** a otra Actividad. Sin embargo, también **puede contener datos** para que la Actividad de destino los utilice (como detalles sobre un ítem a ser mostrado), así como emplearse para **pasar datos de vuelta** a la actividad de origen.

Una *Intent* suele contener dos datos principales:

- Acción a realizar (por ejemplo, ACTION_VIEW, ACTION_EDIT, ACTION_MAIN)
- Datos sobre los que operar (por ejemplo, el registro de una persona en la base de datos de contactos)

Intent implícito

Los Intent implícitos **no especifican un objetivo previsto**, sino que proporcionan información suficiente para que **el sistema resuelva qué componente debe gestionar la llamada**.

- Proporciona una acción genérica que la aplicación puede realizar
- Se resuelve mediante la asignación del tipo de datos y la acción a componentes conocidos.
- Permite que cualquier aplicación que cumpla los criterios gestione la solicitud.

```
fun enviarEmail() {  
    val intent = Intent(Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(Intent.EXTRA_EMAIL, "hola@ejemplo.es")  
    intent.putExtra(Intent.EXTRA_TEXT, "Como estas?")  
  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```



Comprueba que existe una
app capaz de gestionar el
Intent lanzado

Ejercicio 2

1. Crea una aplicación que tenga un campo de texto y un botón.
2. El campo de texto mostrará un *hint* indicando que se debe introducir una URL
3. Al pulsar el botón, la aplicación deberá tomar la URL del cuadro y abrirla a través de un navegador externo (instalado ya en el teléfono).

El Intent deberá crearse utilizando su constructor que recibe:

- Una acción: *Intent.ACTION_VIEW*
- Datos: *Uri.parse(String)* [tipo URI]

Intent explícito

Los Intents explícitos son más estrictos, ya que **indican un componente específico que debe gestionar la solicitud**.

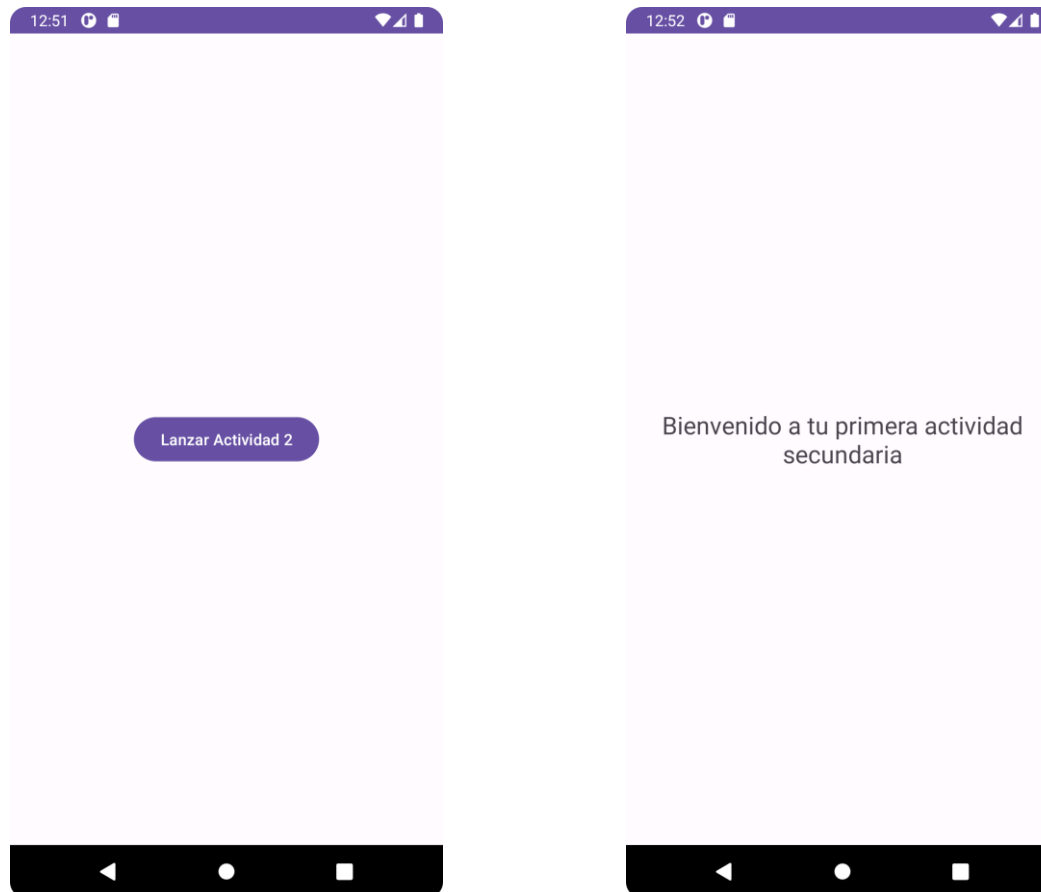
Se utilizan normalmente cuando se **navega entre componentes dentro de la aplicación** y se conoce el nombre de la clase. También puede utilizarlas para navegar a una aplicación conocida de terceros si está absolutamente seguro del tipo de Intent que pueden manejar.

```
fun verDetallesPelicula() {  
    val intent = Intent(this, PeliculaActivity::class.java)  
    intent.putExtra(PELICULA_ID, peli_id)  
    startActivity(intent)  
}
```

La función *putExtra()* nos permite enviar datos en el Intent que pueden ser utilizados en la actividad de destino.

Ejercicio 3

Crea una aplicación con un botón. Al hacer click en este se deberá lanzar una actividad secundaria como se muestra a continuación.



Recoger datos del Intent

Cuando se lanza una actividad a raíz de una interacción del usuario con una vista de nuestra aplicación (ej. un botón) **se pueden enviar datos a través del Intent**.

Para recoger estos datos en la actividad que se crea, se debe acceder a los datos “extra” recibidos en el elemento ***intent*** de la actividad.

```
class SecondaryActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_secondary)  
        val nombre = intent.getStringExtra("NOMBRE_PELICULA")  
        val visionada = intent.getBooleanExtra("VISTA", false)  
        // Recibir otros datos adicionales...  
    }  
}
```

Ejercicio 4

Partiendo de la aplicación desarrollada en el ejercicio anterior:

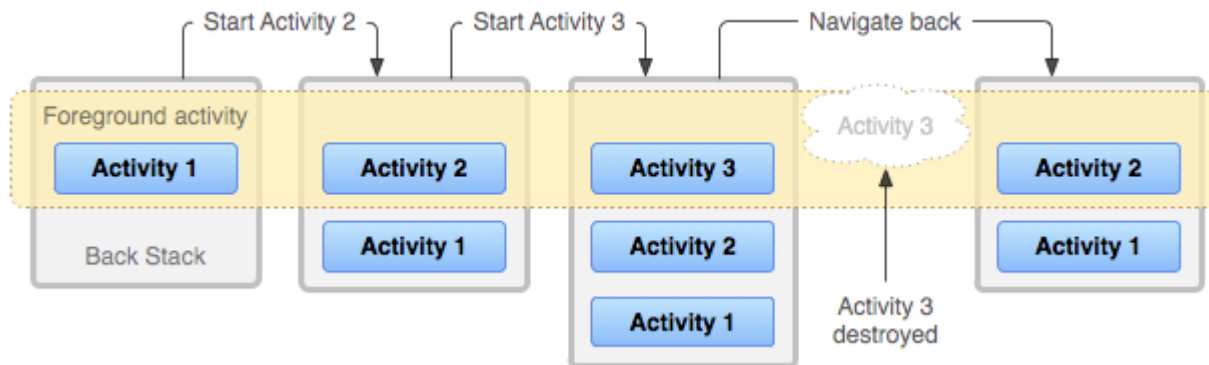
1. Añade un cuadro de texto a la actividad principal
2. Modifica el funcionamiento del botón para que, al pulsarlo, el texto contenido en el cuadro de texto se envíe a la actividad secundaria al lanzarla
3. Añade un campo de texto a la actividad secundaria
4. Modifica la actividad secundaria para mostrar el texto recibido en el campo de texto añadido

Pila de actividades

Pila de actividades o *back stack*

En Android, **las actividades se organizan en una pila (*back stack*)** en el orden en que se abre cada actividad. Es decir, **la última actividad en entrar es la primera en salir (*LIFO: last in, first out*)**.

Ejemplo: en la aplicación de contactos del móvil, la actividad principal muestra la lista de contactos guardados. Cuando pinchamos en uno, se abre otra actividad para ver los detalles. Esta nueva actividad se agrega a la pila. Al navegar hacia atrás, se destruye y se quita de la pila.



Sobrescribiendo la navegación hacia atrás

En ocasiones nos puede interesar **controlar lo que pasa cuando el usuario presiona el botón *Atrás***. Para ello, debemos sobrescribir su funcionamiento por defecto, asociando una *callback* personalizada con la lógica deseada.

```
MyActivity.kt

class MyActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_about)

        // ...
        onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true){
            override fun handleOnBackPressed() {
                TODO("Lógica a realizar")
                finish() // Si queremos cerrar la actividad
            }
        })
    }
}
```

Ejercicio 5

Modifica la aplicación desarrollada en el ejercicio anterior para que cuando el usuario esté en la actividad principal, deba pulsar dos veces el botón atrás antes de salir de la aplicación.

Al pulsar la primera vez, se mostrará un aviso (Toast): *Pulsa una vez más para cerrar la aplicación*

Uso de avisos flotantes Toast

```
Toast.makeText(this | applicationContext, "Mensaje", Toast.LENGTH_SHORT).show()
```

Desde un callback
de la Actividad

Desde un callback
de otro objeto

También se puede
Toast.LENGTH_LONG

**Lanzando actividades
para obtener un
resultado**

Actividades con resultado

Dependiendo de la naturaleza de la aplicación, **podemos tener tareas que deban realizarse en actividades distintas, devolviendo un resultado**. Cuando esto ocurre, la actividad secundaria se debe lanzar de una manera especial para poder capturar su resultado.

Ejemplo: en la aplicación de contactos del móvil, la creación de un contacto nuevo se suele manejar en una actividad independiente. Cuando el usuario introduce los datos o cancela la operación, se vuelve a la actividad principal y se actualiza el listado si es necesario.

Paso 1: lanzador de actividades con callback para manejar el resultado

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    private val getResult = registerForActivityResult(ActivityResultContracts.StartActivityForResult()){
        if (it.resultCode == Activity.RESULT_OK){
            Log.i(TAG, "Recibido ${it.data?.getStringExtra("nombre")}")
        }else{
            Log.i(TAG, "Operación cancelada")
        }
    }
}
```


Actividades con resultado (*cont*)

Paso 2: lanzar el intent de la actividad con el lanzador creado

MainActivity.kt

```
class MainActivity : AppCompatActivity() {  
    private lateinit var btnAcerca : Button  
    // ...  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        btnAcerca = findViewById(R.id.btnAbout)  
        btnAcerca.setOnClickListener {  
            val intent = Intent(this, SecondaryActivity::class.java)  
            getResult.launch(intent) // En sustitución de: startActivity(intent)  
        }  
    }  
}
```

Paso 3: devolver un resultado desde la actividad lanzada

SecondaryActivity.kt

```
class SecondaryActivity : AppCompatActivity() {  
    // ...  
    setResult(Activity.RESULT_OK, Intent().putExtra("nombre", "Javier"))  
    // setResult(Activity.RESULT_CANCELED)  
    finish()  
}
```

Ejercicio 6

Crea una aplicación con dos botones, uno al lado del otro, situados en la parte inferior de la pantalla. Añade un TextView que ocupe el resto de la interfaz.

El botón derecho deberá lanzar una actividad secundaria que permita al usuario, mediante los campos pertinentes, enviar dos valores de vuelta: el nombre de un producto y la cantidad a comprar.

Cuando finalice la operación con éxito, la actividad principal mostrará los datos recibidos en una línea nueva del TextView. Si, por el contrario, el usuario navega hacia atrás, se mostrará un aviso flotante (Toast) indicando que la operación se ha cancelado.

El botón izquierdo servirá para limpiar la lista de la compra y resetear el TextView.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>